INTERGRATE COMPUTING DEVICES

Assessment B

18024092 – Fealy

Contents

Contents 1
Introduction 2
Flow Chart Diagram 3
Implementation 4
Initialisation 4
Website setup 4
Main code 4
Development 5
First Iteration 5
Second iteration 5
Final iteration 6
HTML 6
Testing 6
Conclusion 6
Bibliography 6
Evidence 7
1.1 7
1.2 8
1.3 8
1.4 9
1.5 9
1.6 10
2.1 10
3.1 11

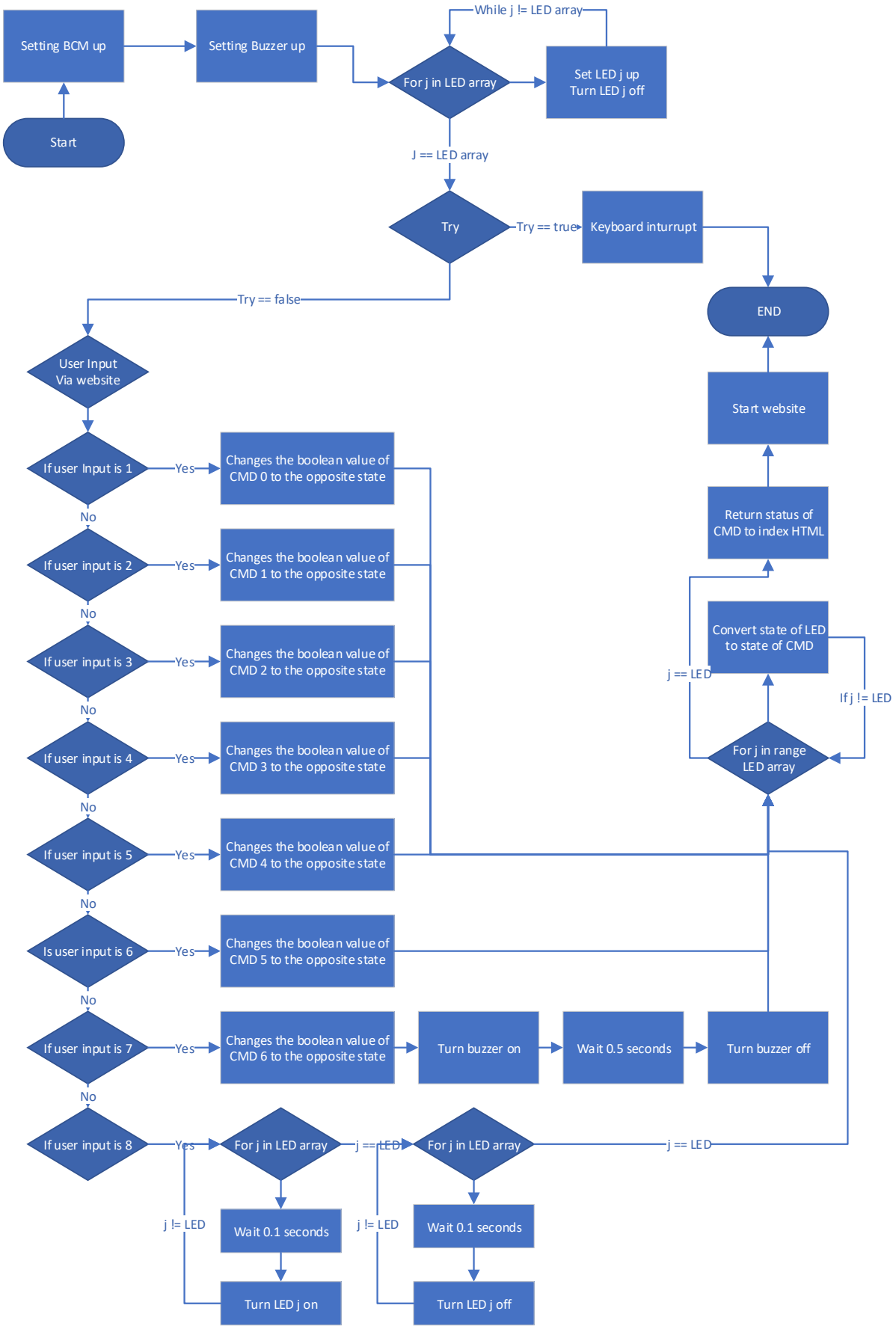
wAssessment A -Dice Game with Raspberry Pi

Introduction

In this assignment the task presented is to create a python and HTML solution to enable the ability for clients over a local area network to active a multitude of buttons and buzzer effects, these effect will be displayed on screen using render updates when the client presses a button, additionally with these effects being shown on the BerryClip(Blog.soton.ac.uk, 2018) attachment. The solution additionally has added a failsafe measurement into the solution in the highly unlikely event of a webpage crash, this failsafe has achieved by using try and excepts, by pressing Ctrl+C. This solution has included added additional functionality that isn't asked for from the specification, these are:

- A cascading style sheet, this improves the ascetics of the website.
- Rainbow effect on the LEDs
- Melody played on the buzzer

Flow Chart Diagram



Implementation

[Complete code under evidence heading]

Initialisation

This section of code demonstrates the codes process for the beginning of the solution, this section of code utilises imports, arrays, global variables, and for loops.

```
#Imports all needed libs
import web
import time
import RPi.GPIO as GPIO

LEDArray = [4, 17, 22, 10, 9, 11] #Sets array for all LEDs
Buzzer = 8 #Stores variable for Buzzer

GPIO.cleanup #Ensure the BCM is clean

GPIO.setmode(GPIO.BCM) #Sets up the BerryClip Board
GPIO.setup(Buzzer, GPIO.OUT) #Sets up buzzer

for i in range(0, 6):   #Sets up all LEDs and then turns them all off
    GPIO.setup(LEDArray[i], GPIO.OUT)
    GPIO.output(LEDArray[i], GPIO.LOW)

global CMD #Sets globals variables
CMD = [0,0,0,0,0,0,0,0]
```

Website setup

This code is responsible for setting up the rendering pathways (Webpy.org, n.d.), URLS, with defining the variables that app with utilise, finally this section demonstrates the statement that allows for the site to be displayed.

```
render = web.template.render('web/')#Sets web page up
urls =('/','index')
app = web.application(urls,globals())

if __name__ == "__main__":
        app.run()
```

Main code

The main section of this code is contained in a definition called Index. This definition is exclusively used for the benefit of the Index HTML document. Additional, webpages could be created by creating another definition and another HTML document. In the index definition is where the Try and except are implemented, additionally this is where all the button inputs from the website is handles. When a button is pressed the CMD status will be inversed then the LED will turn to the opposite state. At the end of this section of code the render status is then passed to the HTML document.

```
#Index class
class index:
    def GET(self):
        while(True):
            try:
                #Handles web input
                getInput = web.input(turn='')    #Handles user web input
                command = str(getInput.turn)     #Turns input to usable string
                if command == "1":               #Following codes determins the user input
                    CMD[0] = (CMD[0] + 1)%2      #Then sets inverses the status of CMD
                if command == "2":
                    CMD[1] = (CMD[1] + 1)%2
                if command == "3":
                    CMD[2] = (CMD[2] + 1)%2
                if command == "4":
                    CMD[3] =(CMD[3] + 1)%2
                if command == "5":
                    CMD[4] =(CMD[4] + 1)%2
                if command == "6":
                    CMD[5] = (CMD[5] + 1)%2
                if command == "7":
                    CMD[6] = (CMD[6] + 1)%2      #Handles buzzer
                    GPIO.output(Buzzer, GPIO.HIGH)  #Turns buzzer on for 0.5 seconds
                    time.sleep(0.5)
                    GPIO.output(Buzzer, GPIO.LOW)   #Then turns buzzer off
                if command == "8":
                    for i in range(0, 6):    #Handles turning all LEDs on
                        GPIO.output(LEDArray[i], GPIO.HIGH)
                        time.sleep(0.1)      #0.1 second between each LED for effect
                    time.sleep(0.1)
                    for i in range(0, 6):    #Turns all LEDs
                        GPIO.output(LEDArray[i], GPIO.LOW)
                        time.sleep(0.1)      #Time delay

                for i in range(0, 6):        #Manages all LED inputs based on their CMD status
                    GPIO.output(LEDArray[i], CMD[i])
                return render.index(CMD)     #Tells HTM1 code the
```

Development

Through the development of this solution required multiple iterations, these iterations allowed the development of CSS and additional features:

First Iteration

In iteration one, the primary goal was to allow for one LED to be switched on in the local host. During this iteration the HTML page was incredible basic with simple CSS to correctly display the button for LED one. The python code in this iteration wasn't optimal and didn't allow for easy additional features. While in this iteration, a few key issues were discovered, these include:

- Need for methods to handle LEDs.
- Possibility to operate over local area networks.
- Allowing for the button function.

Second iteration

The second iteration of the solution initially focused efforts on the key issues found during the first iteration. While during this iteration the functionality to allow the program to operate on a local area network was implemented, an issue occurred while trying to make this feature happen automatically; a temporary solution was constructed which required the HTML document to be update with the Raspberry Pi's IP address prior to the programming being started. During this iteration the possibility for all LEDs to be operated over a local area network, this was done by using

an array called CMD to handle the render state of each LED. At the end of this iteration the code was fully functional and run, however the code was not in its most optimal form, this will be addressed in the final iteration.

Final iteration

The final iteration primarily focused on the editing of the code to be more optimal than the previous generations. The code was optimised by reducing all the index CMD returning at the end of every if statement, instead the index CMD is returned at the end of the method, and in the if statements for each command the Boolean values for the CMD value is switched. The ascetics of the webpage was addressed on this iteration, each button was organised into a list, to create a menu system that allowed for their hyperlink decorations to be adjusted and their styling to be constant. Additional functionality was added in this final stage by adding the option to the user to light all the LEDs up and turn them off in a rainbow-like fashion.

HTML

Testing

Test	On screen result	Real life result	Evidence
Turn on LED 1	“ LED 1 : 1”	First LED coming on	1.1
Turn on LED 2	“ LED 2 : 1”	Second LED coming on	1.2
Turn on LED 3	“ LED 3 : 1”	Third LED coming on	1.3
Turn on LED 4	“ LED 4 : 1”	Forth LED coming on	1.4
Turn on LED 5	“ LED 5 : 1”	Fifth LED coming on	1.5
Turn on LED 6	“ LED 6 : 1”	Sixth LED coming on	1.6
All LEDs off	“ LED 1 : 0”	First LED turning off	
	“ LED 2: 0”	Second LED turning off	
	“ LED 3 : 0”	Third LED turning off	
	“ LED 4 : 0”	Forth LED turning off	
	“ LED 5 : 0”	Fifth LED turning off	
	“ LED 6 : 0”	Sixth LED turning off	
Pressing buzzer button	N/A	Buzzer turning on for half a second then turning off	N/A
Pressing Rainbow mode	N/A	All LEDs turning on, then turning off again	3.1

Conclusion

Before development of this project I had never worked with or used web.py nor ever attempted to develop an online web server platform, however, after completing the solution I now am able to harness the knowledge gathered through the development of this project to further improve the current web page, but also understand the process to create another web server to handle a multitude of tasks.

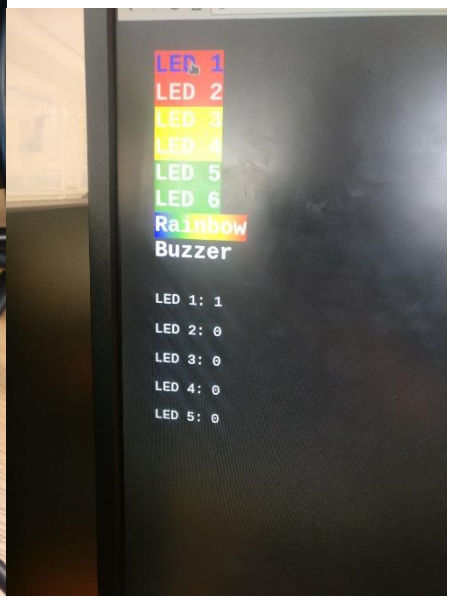
Bibliography

Blog.soton.ac.uk. (2018). *Berry Clip – Raspberry Pi Development Notes*. [online] Available at: https://blog.soton.ac.uk/pi/modules-available/berry-clip/ [Accessed 11 Mar. 2019].

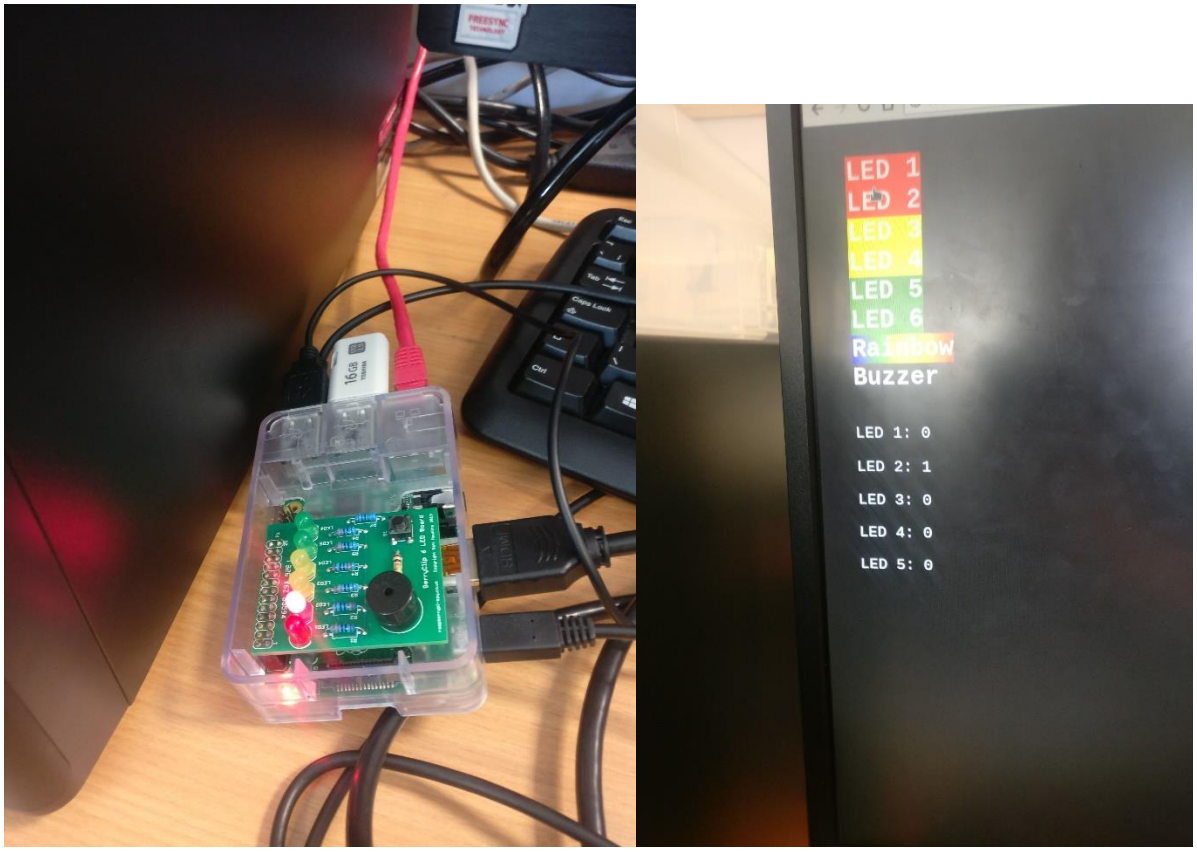
Webpy.org. (n.d.). *Tutorial (web.py)*. [online] Available at: http://webpy.org/docs/0.3/tutorial [Accessed 11 Mar. 2019].

Evidence

1.1



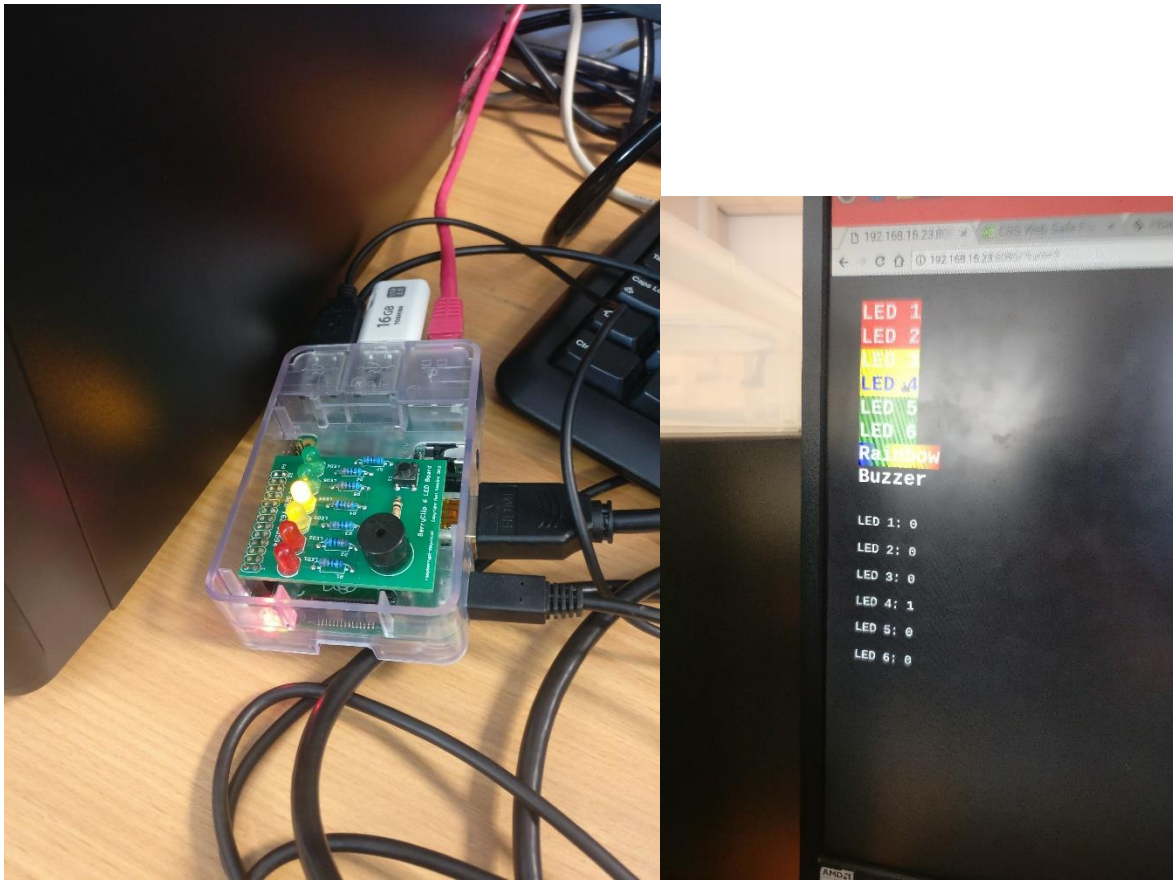
1.2


LED 1
LED 2
LED 3
LED 4
LED 5
LED 6
Rainbow
Buzzer
LED 1: 0
LED 2: 1
LED 3: 0
LED 4: 0
LED 5: 0

1.3

LED 1
LED 2
LED 3
LED 4
LED 5
LED 6
Rainbow
Buzzer
LED 1: 0
LED 2: 0
LED 3: 1
LED 4: 0
LED 5: 0

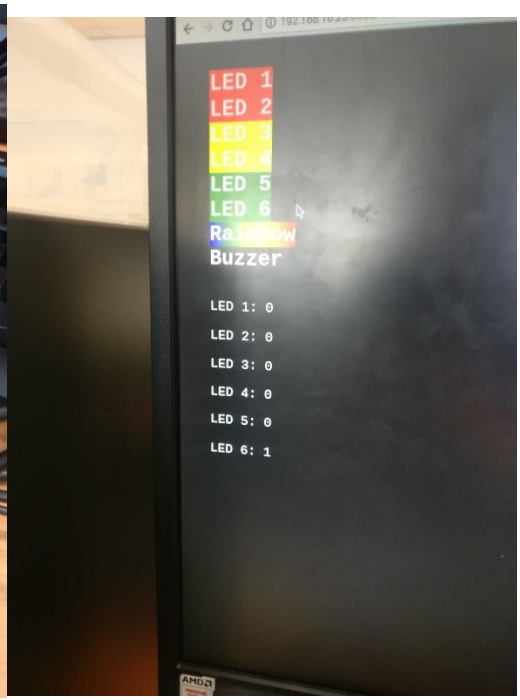
1.4


LED 1
LED 2
LED 3
LED 4
LED 5
LED 6
Rainbow
Buzzer
LED 1: 0
LED 2: 0
LED 3: 0
LED 4: 1
LED 5: 0
LED 6: 0

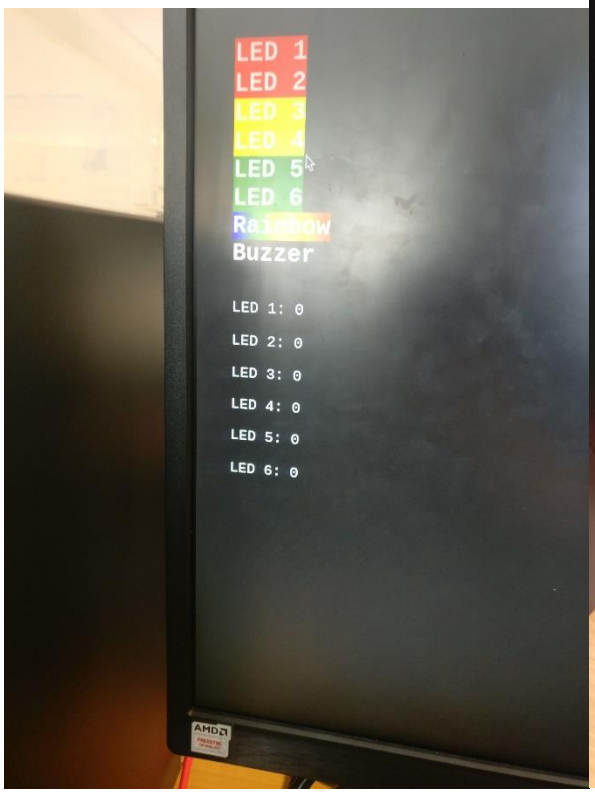
1.5

LED 1
LED 2
LED 3
LED 4
LED 5
LED 6
Rainbow
Buzzer
LED 1: 0
LED 2: 0
LED 3: 0
LED 4: 0
LED 5: 1
LED 6: 0

1.6


LED 1
LED 2
LED 3
LED 4
LED 5
LED 6
Rainbow
Buzzer
LED 1: 0
LED 2: 0
LED 3: 0
LED 4: 0
LED 5: 0
LED 6: 1

2.1


LED 1
LED 2
LED 3
LED 4
LED 5
LED 6
Rainbow
Buzzer
LED 1: 0
LED 2: 0
LED 3: 0
LED 4: 0
LED 5: 0
LED 6: 0

3.1